Artifact-to-Execution Assurance for Canonical Integer GBDTs on the EVM

Formal specification, adversarial conformance, and a pinned provider-attested deployment witness

Mikhail Fedorov

Public technical report v0.2.3
5 September 2026 — not peer reviewed

Abstract

Replicated ledgers can preserve public model bytes and deterministic state transitions, but do not by themselves specify artifact semantics, bind registry identifiers to reconstructed content, or authenticate RPC-returned history. Prior work covers verifiable decision-tree prediction and hash-bound quantized parameters with exact off/on-chain inference. We instead ask whether a registry-named multi-object artifact reconstructs canonically and supplies the same wire-level integer function to local and EVM evaluators. GL1F uses fixed-width encodings and an ordered code-as-data manifest for fixed-depth integer-quantized gradient-boosted trees. We give specification-level sufficient conditions for exact reconstruction, bounded traversal, and conforming-evaluator equality under explicit state, manifest, metadata, input, access, and accumulation premises. Three trainers produced identical cores for 25 distinct profiles plus one standalone IEEE-754 witness; an isolated EVM study matched 72/72 outputs. At GenesisL1 block 13,342,043, a recorded provider-attested summary reports reconstruction of 31,185,324 bytes from 12 records and 108/108 matching local/provider-returned calls. The raw provider responses are not included, so this result is a historical observation rather than independently reproducible evidence. This establishes bounded artifact-to-execution assurance for a developer-operated case study, not predictive validity, privacy, authorship, or cost superiority.

Keywords: gradient boosting; decision trees; fixed-point inference; reproducible computation; EVM; smart contracts; code-as-data; decentralized science

1 Introduction

Reproducible computational research requires more than access to a source dataset or a trained model file. A result may depend on undocumented feature ordering, library versions, parser behavior, floating-point operation order, serialization conventions, or a remote service that later changes. Computational-science and machine-learning communities have consequently treated executable artifacts, environment capture, and explicit evaluation protocols as first-class research objects [27, 28]. A ledger can strengthen one narrow part of this record: it can make public bytes and deterministic state transitions persistent and reviewable at a selected state. It cannot by itself define their canonical semantics, authenticate provider-returned history, or determine whether the underlying scientific question, data, labels, or model are valid.

For arithmetic equality alone, neither a ledger nor a token is necessary: a content-addressed object store plus a canonical evaluator can suffice. The ledger role examined here is narrower. It orders and pins public storage and registry/service state at a block, exposes runtime code through ordinary chain read interfaces subject to RPC and consensus assumptions, and permits contract-mediated evaluation and events. Applications that do not need those properties can use a simpler publication substrate without changing the GL1F integer function.

GBDTs are a useful case for studying this boundary. They are expressive, widely deployed tabular models [6, 14, 21], yet their inference kernel is structurally simple: compare one feature to one threshold, select a child, read one leaf per tree, and add. If thresholds, leaves, inputs, byte order, and branch semantics are fixed as integers, the prediction can be specified without floating-point ambiguity. Fixed-depth complete trees further make the number of branches and serialized offsets explicit.

GL1F turns that observation into an end-to-end public execution artifact. A canonical core begins with a compact header and stores every tree in flat heap order. Version 1 returns a scalar integer score; version 2 returns a vector of integer logits. GL1F serializes additive, fixed-depth integer tree ensembles, and the included trainers construct them by gradient boosting; the core alone does not attest which training algorithm produced it. The same bytes are consumed by a separately implemented local decoder and by a Solidity runtime. Because ordinary EVM contract code is public, chunks used as data can be downloaded without using the canonical inference endpoint. This openness is intentional: GL1F provides transparent, reconstructible models rather than confidential weights.

The system is deployed on GenesisL1, an EVM-compatible network with chain ID 29. The deployed publication layer comprises a chunk store, registry, model NFT, inference runtime, and optional marketplace. Inference may be public, access-controlled, or metered according to mutable registry settings. These service controls do not prevent independent local evaluation of intentionally public model bytes.

The central question addressed by this paper is therefore precise:

> *Under which representation, storage, state, input, and arithmetic conditions do conforming local and EVM evaluators produce exactly the same integer result for a public fixed-depth GBDT?*

The answer is not "because the model is on a blockchain." It is a composition of checkable premises: canonical serialization, exact chunk reconstruction, strict decoding, header/registry consistency, identical little-endian int32 inputs, identical strict-greater traversal, and an accumulator wide enough for the finite integer sum. This paper gives a specification-level proof under those premises, supplies source inspection and finite tests that map the tested implementation to the specification, and evaluates one provider-attested deployment state.

1.1 Contributions

The paper makes the following bounded contributions:

C1. A fixed-width scalar/vector binary representation for complete fixed-depth integer GBDTs, with exact byte-length formulae and normative little-endian inference semantics.

C2. A code-as-data publication scheme and a formal manifest definition under which one-, two-, and four-byte primitives are reconstructed exactly across chunk boundaries. Code-as-data storage itself is established prior art; the contribution is its bounded manifest and its composition with the GL1F format and evaluators.

C3. Proofs of leaf-index safety, finite work, exact equality between conforming local and EVM evaluator specifications, safe JavaScript accumulation for the documented deployment profile, conditional fixed-point output error, and split/argmax stability.

C4. Three first-party trainers plus a separately implemented strict parser, conformance evaluator, malformed-input suite, and cross-engine exact-byte tests.

C5. A reproducible local-EVM integration method that deploys the contracts, publishes v1 and v2 fixtures through 17-byte chunks, and compares separately implemented bigint results with view and transaction endpoints.

C6. A number-pinned public-deployment witness that reconstructs every active model at the selected state, checks content identity and nine explicitly defined registry/core/storage

relations, and compares nine deterministic int32 inputs per model between local and EVM read-call results.

C7. An explicit trust-boundary analysis separating fixed runtime-code evidence at a pinned block from mutable registry, access, NFT, marketplace, RPC, and application-specific scientific assumptions.

No contribution is formulated as a claim of originating GBDTs, model interchange, on-chain machine learning, or verifiable inference. No benchmark against XGBoost, LightGBM, ONNX runtimes, proof systems, or competing blockchains was conducted. The measured trainer timings are engineering reference observations, not a general ranking.

1.2 Claims and evidence classes

Table 1 states the principal claims at the strength supported by the evidence. Proofs concern all objects satisfying explicit premises; tests and chain observations concern only their enumerated inputs and pinned states.

Table 1: Claim-to-evidence map. P: mathematical proof; T: executable test or benchmark; C: pinned chain observation; S: source inspection.

ID	Class	Claim and boundary
C1	P/S	A depth-d canonical tree occupies $12 \cdot 2^d - 8$ bytes; v1 and v2 core lengths follow exactly from the header, output count, and tree count.
C2	P	A canonical traversal makes exactly d decisions and selects one in-range leaf; v1 performs Td and v2 performs KRd decisions.
C3	P/S/T/C	Conforming local and EVM evaluator specifications return the same integer result given externally verified canonical bytes, a well-formed same-state storage manifest, matching metadata, identical packed input, strict comparison and argmax semantics, satisfied endpoint access conditions, and exact accumulation. Source inspection and finite tests, rather than the proof alone, support the deployed `ForestRuntime` path.
C4	P	For the abstract exact-real quantizer, without saturation and with unchanged paths, scalar dequantization error is at most $(T+1)/(2Q)$; $\lvert x-\tau\rvert > 1/Q$ is sufficient to preserve one split. Production binary64 preprocessing requires the stated product-rounding correction.
C5	T	The tested JavaScript, Python, and C++ trainers emitted identical core bytes across 25 distinct training profiles under the documented finite-input publication profile; a separate arithmetic witness exercises the IEEE-754 operation-order boundary. This is not a platform-universal theorem about training.
C6	T	The local EVM suite reconstructed deliberately boundary-spanning v1 and v2 fixtures and exactly matched separately implemented scalar, vector, class, and tie case outputs.
C7	C/T	The recorded provider-attested summary for block 13,342,043 reports that all 12 active records passed reconstruction, commitment, and nine named registry/core/storage relations, with 108/108 matching local/provider-returned calls. The raw responses are not included, and the observation does not validate the models' datasets or predictive quality.
C8	S/P	Model bytes are intentionally public and permit independent local inference. Pricing controls the canonical endpoint or service path, not confidentiality or exclusive possession of weights.

2 Related work and design position

2.1 Gradient boosting and model interchange

Gradient boosting constructs an additive predictor by iteratively fitting weak learners to a loss-derived target [14]. XGBoost adds a regularized objective, sparsity-aware algorithms, and scalable system design [6]; LightGBM emphasizes histogram construction, leaf-wise growth, feature bundling, and distributed efficiency [21]. Those systems are mature choices for large-scale training and predictive-performance studies. GL1F does not attempt to replace them. It narrows the model family to fixed-depth complete trees so that serialized offsets and consensus-facing work are statically transparent.

ONNX provides a broad graph intermediate representation for models and operators [25]. Its `ai.onnx.ml` `TreeEnsemble` operator already specifies batched tree-ensemble inference, branch modes, leaf targets, aggregation, and post-transforms [26]. Hummingbird translates conventional machine-learning pipelines to tensor computations for optimized prediction serving [24]. These systems have broader operator and runtime surfaces than the integer tree kernel examined here. GL1F's narrower grammar trades ecosystem breadth for fixed-width offsets, a small strict parser, and direct EVM interpretation; it is not a priority claim for tree interchange or portable prediction.

2.2 Blockchain-hosted machine learning

Blockchain-hosted machine learning predates this study in several forms. Harris and Waggoner proposed an Ethereum framework for a publicly hosted, continuously updated model with collaborative contributions and incentives [18]. Drungilas et al. compared logistic-regression inference in Hyperledger Fabric chaincode with inference delegated to an oracle service [11]. Badruddoja et al. implemented integer-based, probability-oriented Naive Bayes prediction in Solidity and evaluated it against a Python reference [3]. Kadadha et al. trained bagged-tree behavior models off-chain and deployed prediction logic as smart contracts in a blockchain crowdsourcing system [20]. ML2SC translates PyTorch multilayer perceptrons to Solidity, transfers trained parameters, uses fixed-point arithmetic, and models deployment, update, and inference gas costs [23].

Prior peer-reviewed work also demonstrates bit-exact, gas-bounded on-chain decision-tree inference and formal verification in a decentralized-learning application. Most directly, Alhaidari et al. report quantized model parameters serialized for propagation, cryptographic hash binding between L2 and L1, read-only `eth_call` and state-changing inference tiers, gas analysis, and Z3-checked bit-exact off/on-chain inference, including decision trees [2]. This is substantial property-level overlap, not merely a neighboring application. GL1F therefore makes no priority claim for on-chain machine learning, serialized or hash-bound quantized parameters, off-chain training followed by contract deployment, fixed-point inference, read-call inference, or exact integer tree execution.

Fu et al. authenticate binary decision-tree predictions using Merkle-tree and hash commitments, provider-supplied path proofs, and a verification procedure, and support incremental tree updates [15]. Their object is a proof that a service result follows a committed tree and sample; GL1F instead exposes public ensemble bytes and checks exact reconstruction plus direct evaluation. At a different scaling point, opML moves general ML execution off-chain and uses an interactive, optimistic fraud-proof protocol with on-chain dispute resolution [8]. GL1F neither provides an optimistic dispute game nor inherits its challenge/liveness assumptions; it directly interprets a bounded public tree format locally and in the EVM. The cited systems study different assurance properties, models, and deployment assumptions; no cross-system performance ranking was performed here.

2.3 EVM semantics and verification

The EVM is a replicated deterministic state machine whose instruction and storage costs are exposed through gas [35]. It has no native floating-point arithmetic, and deployed runtime code is constrained by the 24,576-byte EIP-170 limit [5]. Using contract runtime code as immutable data, written with `CREATE` and read with `EXTCODECOPY`, predates GL1F and is available in libraries such as SSTORE2 [1]. GL1F does not claim this storage primitive as novel. The evaluated construction is a multi-chunk, table-addressed manifest with exact framing and length conditions, joined to a specified tree representation and two evaluation paths.

KEVM gives an executable formal semantics of EVM bytecode [19]. The equivalence theorem in this paper is specification-level: it relates evaluators satisfying explicit decoding, comparison, indexing, and accumulation premises. The deployed `ForestRuntime` bytecode was not encoded in KEVM, and the theorem is not a mechanized bytecode proof. Local-EVM tests and the pinned-chain witness are empirical evidence under their enumerated fixtures and state.

Proof-based verifiable ML takes another route. A prover evaluates a model off-chain and supplies a proof that a circuit was executed correctly; weights or data may remain hidden depending on the construction. Prior systems cover random-forest classification on certified client data, zero-knowledge decision-tree prediction and accuracy, and verifiable decentralized decision-tree pipelines [9, 31, 36]. Broader zkML systems extend this design space to other model families [33]. GL1F directly executes public weights and does not require a per-inference prover. Direct execution is transparent for bounded models but provides no weight privacy and incurs replicated execution. Proof systems change those privacy and cost relations while adding circuit, prover, setup, and proof-generation considerations. GL1F does not improve those proof systems, and neither architecture dominates all use cases.

2.4 Provenance and prior documentation

Software- and model-provenance systems set a broader context for the artifact-to-execution argument. in-toto authenticates authorized software-supply chain steps and their material/product links [34]. MLflow2PROV captures PROV-compliant relationships among data, model, metadata, software, and pipeline activities [29]. This release implements neither an in-toto layout nor a general provenance graph. Its hashes, manifests, tests, and pinned observations establish narrower relations from a specific byte string to evaluated execution; they do not establish dataset lineage, trainer identity, or predictive validity. Likewise, a pinned compiler profile is not by itself a reproducible-build result in the supply-chain sense [22]: the release does not claim independent, byte-identical rebuilds or signed builder-generated provenance. Sentry addresses a complementary supply-chain property: it ties developer identities to signatures and authenticates model and dataset artifacts while they are loaded into GPU memory using accelerated hash constructions [16]. GL1F does not authenticate an artifact's author or origin; its hashes establish byte identity only under their stated binding premises.

A public GL1F protocol and technical document dated 19 January 2026 already described the alpha architecture, binary formats, code-as-data storage, contract interfaces, and browser workflow [10]. That document is prior system documentation, not evidence newly produced by this study; the format category, storage architecture, contracts, and UI therefore are not contributions first introduced by this paper. New here is a versioned evaluation with strict canonical parsing, sufficient ordered-manifest exactness conditions, a specification-level conforming-evaluator equivalence theorem, malformed-artifact and cross-language conformance tests, an executable registry non-binding counterexample, and a provider-attested number-pinned deployment witness summary. Public v0.2.3 includes the method and summarized findings but not the raw provider responses or reconstructed deployed bytes.

A hash can identify bytes; a token can identify an on-chain record; consensus can order transactions. These are different propositions. A content commitment says nothing by itself

Table 2: Evidence-scoped positioning against representative prior work. The middle column records the cited source's reported evaluation object; the final column delimits the object evaluated here, rather than asserting that the prior work lacks other capabilities.

Reference and setting	Reported object	Boundary to this study
ONNX `TreeEnsemble`[26]	Portable tree-ensemble operator semantics over batched tensor inputs.	GL1F studies a smaller fixed-width integer serialization interpreted by an EVM contract and a strict local evaluator.
Harris–Waggoner, Ethereum[18]	Collaborative data/model updating, public inference, and contributor incentives.	The evaluated claim here is assurance for a frozen content-addressed artifact, not collaborative learning incentives.
Drungilas et al., Fabric[11]	Logistic-regression inference in chaincode and through an oracle, with runtime comparison.	GL1F's evaluated execution path is direct public EVM interpretation; no oracle comparison is reported.
Badruddoja et al., Ethereum[3]	Integer approximation for Solidity Naive Bayes prediction and comparison with a Python implementation.	GL1F evaluates fixed-depth integer forests and exact artifact reconstruction, not Naive Bayes probability approximation.
Kadadha et al., smart-contract crowdsourcing[20]	Off-chain-trained bagged trees used for transparent on-chain behavior prediction and task allocation.	This case study tests format, storage, registry relations, and execution identity; it makes no task-allocation claim.
Fu et al., hybrid blockchain[15]	Merkle/hash-committed binary decision tree, provider-generated prediction-path proofs, client verification, and efficient model updates.	GL1F directly reconstructs and evaluates public ensemble bytes; it supplies no private service-result proof and does not improve VDT.
ML2SC, EVM[23]	PyTorch-to-Solidity translation for MLPs, fixed-point inference, and gas-cost models.	GL1F's object is a tree byte format plus an interpreter; no MLP translation or cross-system gas ranking is claimed.
Alhaidari et al., EVM[2]	Serialized and hash-bound quantized parameters, `eth_call` and transaction inference, gas analysis, and Z3-checked bit-exact off/on-chain inference including decision trees.	GL1F's delta is the strict multi-object artifact-to-execution predicate, registry counterexample, and provider-attested observation of one pinned multi-model deployment—not first exact EVM tree inference.
opML, optimistic ML[8]	Native off-chain ML execution with interactive fraud-proof arbitration and on-chain resolution of a disputed step.	GL1F directly interprets a bounded public model and has no challenger or dispute-window security assumption.
Sentry, GPU artifact authentication[16]	Developer-signed datasets/models authenticated during GPU loading with accelerated hashing.	GL1F checks bytes, storage relations, and evaluator behavior; it neither binds developer identity nor authenticates artifact origin.
Public GL1F protocol document[10]	Alpha design, formats, contracts, and operational documentation.	This paper's delta is bounded specification proofs, adversarial tests, traceability, and a provider-attested deployment observation.

about who trained a model, whether metadata are accurate, or whether a score generalizes. Likewise, an NFT transfer under EIP-721 semantics [12] does not automatically transfer scientific authorship, intellectual-property rights, access keys, or a fee destination. GL1F's model NFT and marketplace are discovery and service layers surrounding the execution artifact, not substitutes for dataset provenance, licensing analysis, or independent model evaluation.

3 System architecture

3.1 Components

The reference implementation contains five layers:

Training.
A Web Worker supports an interactive browser workflow. Python and C++17 command-line trainers support reproducible scripts and native execution. All implement regression, binary classification, multiclass classification, and multilabel classification, with fixed-depth histogram-based trees.

Core format.
The `GL1F` core is the inference-relevant byte string. Optional `GL1X` JSON metadata may accompany a core but is explicitly excluded from its content identity and inference semantics.

Immutable storage.
`ModelStore.write` deploys runtime code `GL1C`‖payload. A second such object contains an ordered table of chunk addresses. Payload capacity is 24,572 bytes because the four-byte prefix occupies the remainder of the EIP-170 limit.

Registry and asset services.
`ModelRegistry` records a model identifier, storage shape, inference parameters, settings, access plans, accepted policy versions, and a token mapping. `ModelNFT` supplies ownership and custom metadata; `ModelMarketplace` supplies approval-based listings. These records are mutable or transferable in ways the core bytes are not.

Execution and verification.
`ForestRuntime` reads primitives from chunks with `EXTCODECOPY` and evaluates v1 or v2 trees. Independently, a strict local parser reconstructs and validates the core, computes its hash, and evaluates identical packed inputs. A live witness performs these steps at one block tag.

3.2 Canonical publication flow

Here Ethereum Keccak-256 means the legacy Ethereum hash, not FIPS SHA3-256; keccak256(B) denotes its digest of B.

A publication-grade workflow performs the following operations:

1. freeze the numeric matrix, target representation, row/feature/output ordering, preprocessing, hyperparameters, seed, trainer revision, and execution environment;
2. train and serialize a GL1F core;
3. strictly parse the core and reject unknown versions, nonzero reserved fields, invalid dimensions, nonpositive scale, inconsistent length, or out-of-range feature indices;
4. compute both SHA256(B) for cross-tool artifact reporting and keccak256(B) for the deployed model identifier;
5. divide B into ordered payload chunks, deploy those chunks, and deploy the address table;
6. register the declared storage shape and model settings;
7. independently read the code back at a pinned block, reconstruct exactly the declared length, repeat all strict checks, reconcile the nine registry/core/storage relations later enumerated in Definition 4.10, and verify keccak256(B) = modelId; and
8. archive representative packed inputs and integer outputs from both local and EVM evaluators.

Steps 3 and 7 are essential for the presently deployed registry. Registration accepts the identifier and storage metadata as separate caller-supplied arguments; it does not reconstruct and hash arbitrary model bytes. Registry existence is therefore not, by itself, a proof of content binding.

3.3 Public-byte and service boundary

GL1F stores model bytes as public runtime code. Any observer with ordinary chain-read access can recover the address table and chunks and run the model locally. An optional paid mode can meter the official contract endpoint, generate a canonical event trail, or participate in a service integration; it cannot make public bytecode confidential.

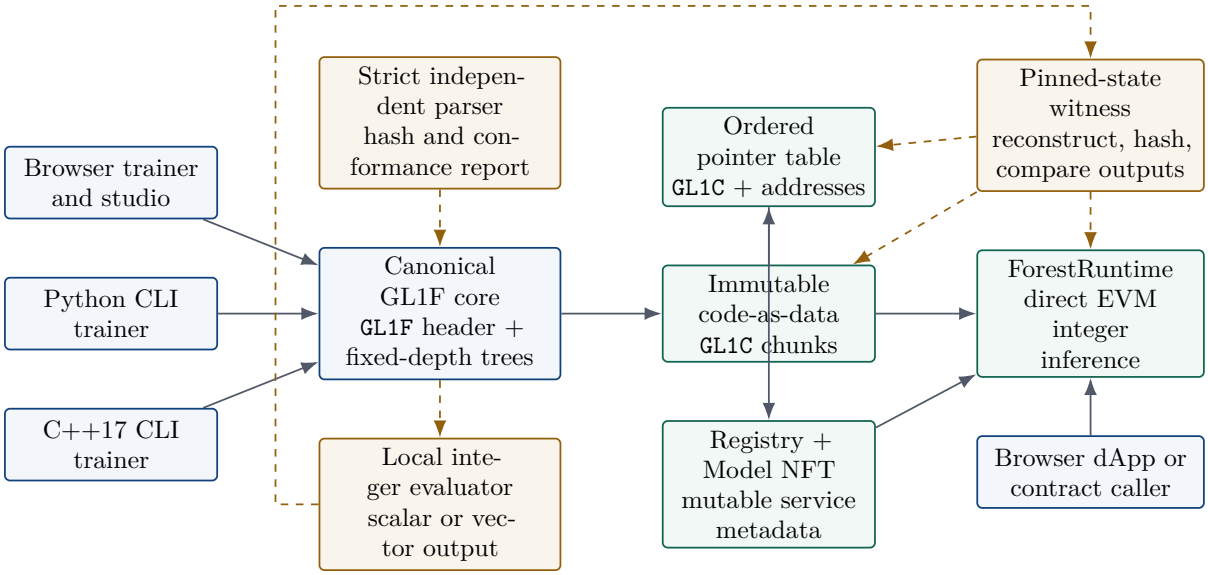

Figure 1: GL1F publication and inference path. Solid arrows show production data flow; dashed arrows show separate verification paths. Model bytes are intentionally public. Registry, NFT, pricing, and marketplace state are services around the model and do not alter its mathematical inference semantics.

4 Formal model and correctness

This section distinguishes the mathematical forest f, its byte representation B, and a registry record that refers to a storage manifest. Exactness of f from B does not authenticate the registry record. The latter implication requires the additional manifest and content-identity premises stated below.

4.1 Canonical core representation

Let

$$\mathbb{I}_{32} = [-2^{31}, 2^{31} - 1] \cap \mathbb{Z}.$$

A complete binary tree of depth d contains

$$I(d) = 2^d - 1 \quad \text{internal nodes}, \qquad L(d) = 2^d \quad \text{leaves}.$$

One internal node occupies eight bytes: a little-endian unsigned 16-bit feature index, a little-endian signed 32-bit threshold, and two reserved bytes. One leaf occupies four bytes containing a little-endian signed 32-bit increment. Hence

$$b(d) = 8I(d) + 4L(d) = 12 \cdot 2^d - 8 \tag{1}$$

bytes are necessary and sufficient for one serialized complete tree.

Definition 4.1 (Canonical v1 scalar core)**.** A canonical v1 core consists of a 24-byte header followed by T trees. It has feature count $F \geq 1$, first-party deployment depth $1 \leq d \leq 12$, unsigned 32-bit wire tree count $0 \leq T \leq 2^{32} - 1$, base $a_0 \in \mathbb{I}_{32}$, and scale $Q \in [1, 2^{32} - 1] \cap \mathbb{Z}$. Its

magic is `GL1F`, version is one, reserved fields are zero, every node feature lies in $\{0, \dots, F-1\}$, and its exact length is

$$|B| = 24 + Tb(d). \tag{2}$$

The wire format admits $T = 0$ as a base-only object; trained publication artifacts use $T \geq 1$. Publication through the deployed registry additionally requires $T \leq 2^{16} - 1$, because its total-tree-count field is unsigned 16-bit. This registry bound is not the v1 wire bound.

Definition 4.2 (Canonical v2 vector core)**.** A canonical v2 core has $F \geq 1$, $1 \leq d \leq 12$, $K \geq 2$ outputs, $R \geq 1$ trees per output, base vector $(a_{0,0}, \dots, a_{0,K-1}) \in \mathbb{I}_{32}^{K}$, positive scale Q, and output-major tree order. Its magic, reserved fields, and feature-index conditions match Definition 4.1, its version is two, and

$$|B| = 24 + 4K + KRb(d). \tag{3}$$

Canonical publication through the deployed registry additionally requires $KR \leq 2^{16} - 1$, because that interface records the total tree count in a 16-bit field. This deployment constraint is narrower than the core format's separate 16-bit K and 32-bit R fields.

Under ordinary hash assumptions, keccak256(B) binds the canonical byte string B; it is syntactic artifact identity, not a semantic normal form. Distinct canonical cores can denote the same extensional integer function. The same v1 scalar bytes may be interpreted by application metadata as a regression score or a binary logit. The same v2 vector may represent multiclass logits or independent multilabel logits. Feature names, units, missing-value rules, calibration, task labels, datasets, and intended use are not encoded by the core. They cannot change the integer function defined by the core, but they are indispensable to scientific interpretation. In particular, the core has no task discriminator: keccak256(B) does not bind whether a v1 score is regression or a binary logit, whether a v2 vector is multiclass or multilabel, or any preprocessing or dataset claim. A publication requiring semantic identity must therefore authenticate a separate semantic manifest.

4.2 Chunked code-as-data

Let c be the nominal payload of each nonfinal chunk and therefore the logical byte-offset stride; the final payload may be shorter. With $\ell = |B|$ and $N = \lceil \ell/c \rceil$, chunk j has runtime code

$$C_j = \texttt{GL1C} \,\|\, B[jc : \min\{(j+1)c, \ell\}].$$

The pointer object contains `GL1C` followed by N 32-byte words; the low 20 bytes of word j hold the EVM address of C_j.

Definition 4.3 (Well-formed storage manifest)**.** For an EVM state σ and registry record r in that state, a tuple $M = (P, c, N, \ell)$ is well formed for B at (σ, r) if:

1. $P \neq 0$, $4 \leq c \leq 24572$, $1 \leq N \leq 767$, $\ell = |B|$, and $N = \lceil \ell/c \rceil$;
2. the code in σ at table address P is exactly $4 + 32N$ bytes, starts with `GL1C`, and contains exactly N address slots;
3. every slot has zero high 12 bytes and resolves through its low 20 bytes to a nonzero chunk address;
4. each nonfinal chunk in σ is exactly $\texttt{GL1C}\|B[jc : (j+1)c]$, and the final chunk is exactly $\texttt{GL1C}\|B[(N-1)c : \ell]$, with no undeclared trailing payload; and
5. the corresponding fields of r equal P, c, N, ℓ.

Lemma 4.4 (Pointer capacity)**.** *One pointer table created by the deployed storage contract contains at most* $\lfloor 24572/32 \rfloor = 767$ *addresses.*

Proof. `ModelStore.write` accepts at most 24,572 payload bytes and prefixes four bytes of runtime magic. Each table entry occupies 32 payload bytes. □

For a manifest satisfying Definition 4.3, define the specified boundary reader $\mathrm{Read}_{\sigma,M}(o,n)$ to compute $j = \lfloor o/c \rfloor$ and $r_o = o \bmod c$, copy from code offset $4 + r_o$ in chunk j, and, only when $r_o + n > c$, concatenate the remaining bytes from code offset four in chunk $j+1$.

Lemma 4.5 (Exact primitive reconstruction)**.** *Under Definition 4.3, for $o \in \mathbb{Z}_{\geq 0}$, $n \in \{1,2,4\}$, and $o + n \leq |B|$, $\mathrm{Read}_{\sigma,M}(o,n) = B[o : o+n]$.*

Proof. Write $o = jc + r$, with $j = \lfloor o/c \rfloor$ and $0 \leq r < c$. If $r + n \leq c$, the reader copies n bytes from code offset $4 + r$ of chunk j, which is exactly the requested slice. Otherwise it copies $c - r$ bytes from the end of C_j and $n - (c - r)$ from the payload start of C_{j+1}. Because $n \leq 4 \leq c$, no third chunk is necessary. Definition 4.3 supplies the exact chunk order, content, and length; concatenation therefore yields $B[o : o+n]$. □

The four-byte `GL1C` prefix is a framing check rather than an authenticator. Arbitrary bytecode can begin with the same magic. Exact reconstruction follows from the entire manifest predicate, not from the prefix alone.

4.3 Traversal, termination, and space

Internal nodes and leaves use heap indexing. Set $i_0 = 0$. At level j, the selected feature value q_f and node threshold τ_q determine

$$i_{j+1} = \begin{cases} 2i_j + 2, & q_f > \tau_q, \\ 2i_j + 1, & q_f \leq \tau_q. \end{cases} \tag{4}$$

Equality goes left; this convention is part of the serialized function.

Lemma 4.6 (Traversal interval)**.** *After j decisions,*

$$2^j - 1 \leq i_j \leq 2^{j+1} - 2.$$

Proof. At $j = 0$, both bounds equal zero. Suppose the interval holds at j. The smallest child is $2(2^j - 1) + 1 = 2^{j+1} - 1$, and the largest is $2(2^{j+1} - 2) + 2 = 2^{j+2} - 2$, which are the claimed bounds for $j + 1$. □

Corollary 4.7 (Leaf safety)**.** *After exactly d decisions,*

$$0 \leq i_d - I(d) \leq L(d) - 1.$$

Thus a traversal of a well-formed tree selects exactly one in-range leaf.

Proof. Substitute $j = d$ in Lemma 4.6 and subtract $I(d) = 2^d - 1$. □

Lemma 4.8 (Forced-subtree inertness)**.** *If a terminal node denoting signed int32 value v is replaced by a complete forced continuation whose remaining internal nodes use feature zero and threshold $2^{31} - 1$, and whose descendant leaves all contain that same v, then the continuation returns v for every $q \in \mathbb{I}_{32}^F$. The replacement therefore preserves the terminal value over the entire packed-input domain.*

Proof. Because $F \geq 1$, feature zero exists. Every int32 input satisfies $q_0 \leq 2^{31} - 1$, so strict greater is false at each inserted node and the traversal repeatedly selects the left child. The reached leaf contains v. The duplication of v at every descendant leaf independently makes the returned value invariant to which descendant is reached. □

Theorem 4.9 (Finite work and logical traversal state)**.** *Canonical v1 inference performs exactly Td node decisions and T leaf reads. Canonical v2 inference performs exactly KRd decisions and KR leaf reads. An abstract streaming evaluator that does not buffer the model requires $O(1)$ scalar logical traversal state and $O(K)$ vector logical traversal state, excluding input storage and implementation-specific read buffers.*

Proof. Every complete tree executes Equation 4 exactly d times and then reads one leaf. The v1 outer loop visits T trees; v2 visits R trees for each of K outputs. No traversal is recursive or conditioned on an unbounded model value. A streaming evaluator stores a scalar accumulator or a length-K result vector together with a constant number of indices and decoded primitive values. □

This is a work-count theorem, not a gas-affordability theorem. A finite encoded model can still exceed a block or transaction gas limit. Model size, storage topology, cache behavior, and execution environment must be measured separately. In particular, the deployed Solidity boundary-read path allocates a fresh dynamic byte buffer for each primitive that spans two chunks. Because EVM memory allocation is monotonic within a call, its transient memory may grow linearly with the number of boundary-spanning reads even though each individual read contains at most 32 bytes. This implementation detail does not change the exact decision and leaf-read counts.

4.4 Exact integer inference equivalence

For scalar B, define

$$f_B(q) = a_0 + \sum_{t=0}^{T-1} v_{t,\pi_t(q)}, \tag{5}$$

where $\pi_t(q)$ is the leaf selected by Equation 4. For output k of a vector model,

$$f_{B,k}(q) = a_{0,k} + \sum_{t=0}^{R-1} v_{k,t,\pi_{k,t}(q)}. \tag{6}$$

A *conforming forest evaluator* decodes the specified signed and unsigned little-endian fields, uses strict-greater traversal, accumulates the mathematical sums without overflow or precision loss, and, when producing a class result, selects the lowest index among equal maximum logits. A conforming local evaluator first validates the version-specific canonical definition. A *conforming EVM evaluator* at (σ, r) obtains every core primitive through $\mathrm{Read}_{\sigma,M}$, uses the version-aware fields in Definition 4.10, and otherwise applies the same forest semantics. These are specification definitions. Source inspection and finite contract tests separately assess whether the deployed `ForestRuntime` path conforms to them. Endpoint-specific enabled, authorization, and fee conditions must be satisfied so execution reaches the internal evaluator with the unchanged packed input.

Definition 4.10 (Registry/core/storage agreement)**.** For the same record r and manifest M used in Definition 4.3, the nine relations reported by the live verifier are

$$\begin{aligned}
nFeatures &= F, & nTrees &= \begin{cases} T, & \text{v1}, \\ KR, & \text{v2}, \end{cases} \\
depth &= d, & baseQ &= \begin{cases} a_0, & \text{v1}, \\ 0, & \text{v2}, \end{cases} \\
scaleQ &= Q, & tablePtr &= P, \\
totalBytes &= \ell, & numChunks &= N, \\
chunkSize &= c. & &
\end{aligned}$$

For v2, $baseQ = 0$ checks the deployed registry convention against the zero-reserved scalar-base header field; the actual base vector follows the fixed header. The first five equalities reconcile runtime registry metadata with the decoded core. The last four reconcile the runtime and byte-information registry getters with the manifest.

Theorem 4.11 (Specification-level conforming-evaluator equivalence)**.** *Let B satisfy Definition 4.1 or 4.2; let its EVM storage at (σ, r) satisfy Definition 4.3; let Definition 4.10 hold; and let both evaluators receive the same $4F$-byte sequence formed by packing $q \in \mathbb{I}_{32}^{F}$ as signed little-endian int32 values. Then conforming local and EVM evaluators return $f_B(q)$ for v1 and $(f_{B,k}(q))_{k=0}^{K-1}$ for v2. Their v2 class operation returns*

$$\left(\min \arg\max_{0 \le k < K} f_{B,k}(q),\ \max_{0 \le k < K} f_{B,k}(q) \right),$$

where the minimum implements the specified lowest-index tie rule.

Proof. Lemma 4.5 gives identical primitive bytes to both evaluators. Hence they decode the same feature indices, signed thresholds, leaves, and base terms. The conforming v1 EVM evaluator obtains registry inference fields instead of parsing all header fields; the explicit agreement premise makes those values identical to the local decoder's values.

At the root of each tree, both evaluators compare the same q_f and τ_q with the same predicate. If their indices agree before one level, Equation 4 gives the same child. Induction over d levels and Corollary 4.7 yield the same leaf. Repeating for every tree gives the same finite sequence of signed integer summands. Exact accumulation then makes Equations 5 and 6 identical. Applying the same lowest-index argmax scan to the identical v2 vector gives the same class index and maximum integer logit. □

Definition 4.10 is a sufficient verification predicate, not a minimal operational premise. For an already packed input, fields not consumed by a selected evaluator remain important deployment-identity checks even when they are not required by that evaluator's arithmetic. This handwritten theorem concerns the two conforming evaluator specifications in one state and record. Source inspection supplies the mapping claim for the Solidity path; compiled tests and pinned observations supply finite implementation evidence. None is a mechanized EVM-bytecode proof, verified Solidity-to-bytecode refinement, or source-to-live-bytecode provenance proof.

Proposition 4.12 (Exact JavaScript sums in the documented deployment profile)**.** *The largest inference accumulators permitted jointly by the deployed registry and one canonical `ModelStore` pointer table are strictly within JavaScript's exact integer range.*

Proof. For v1, the registry's 16-bit tree-count bound gives

$$|f_B(q)| \le (65535 + 1)2^{31} = 2^{47} < 2^{53}.$$

For v2 and $d \ge 1$, Equation 1 gives $b(d) \ge 16$. Lemma 4.4 bounds core payload by $767 \cdot 24572 = 18,846,724$ bytes. Since $K \ge 2$,

$$R < \frac{18,846,724}{2 \cdot 16} < 589,000.$$

Therefore each output and partial sum has magnitude below $(589001)2^{31} < 2^{51} < 2^{53}$. Every int32 input and summand is also exactly representable by a JavaScript `Number`. □

The proposition does not license a decoder to accept arbitrary GL1F-like files of unbounded size. The hardened local decoder separately caps core size and depth before allocating or traversing.

4.5 Fixed-point approximation and decision stability

For finite $z \in \mathbb{R}$ and integer $Q > 0$, define the abstract exact-real quantizer with JavaScript's half-toward-positive-infinity tie rule

$$\mathcal{Q}_Q(z) = \mathrm{clamp}_{\mathbb{I}_{32}}\left(\left\lfloor Qz + \frac{1}{2} \right\rfloor\right). \tag{7}$$

This includes the asymmetric negative-half convention; for example, $\mathcal{Q}_1(-1.5) = -1$.

Equation 7 is a mathematical abstraction: production JavaScript first forms the binary64 product $\mathrm{fl}(Qz) = Qz + \delta_Q(z)$ and then applies `Math.round`. The abstract and production integers agree whenever that product-rounding perturbation does not cross an integer decision boundary. Provided the rounded production value remains in the int32 range, the implementation error bound is

$$\left| \frac{\mathrm{Math.\,round}(\mathrm{fl}(Qz))}{Q} - z \right| \le \frac{1/2 + |\delta_Q(z)|}{Q},$$

not necessarily exactly $1/(2Q)$. If int32 saturation occurs, neither this bound nor the abstract half-unit bound applies.

Lemma 4.13 (Abstract single-value error)**.** *If Equation 7 does not saturate, then*

$$\left| \frac{\mathcal{Q}_Q(z)}{Q} - z \right| \le \frac{1}{2Q}.$$

Proof. For $u = Qz$, $\lfloor u + 1/2 \rfloor \in [u - 1/2, u + 1/2]$. Divide by positive Q. □

Theorem 4.14 (Stable-path output error)**.** *Consider an ideal scalar forest whose base and the T leaves selected for input x are independently quantized by Equation 7 without saturation at common scale Q. If no selected path changes, the dequantized GL1F output $\widehat{y}$ and the corresponding unquantized sum y satisfy*

$$|\widehat{y} - y| \le \frac{T + 1}{2Q}.$$

For one v2 output, replace T with R.

Proof. Apply Lemma 4.13 to the base and every selected leaf and use the triangle inequality. □

Lemma 4.15 (Sufficient split margin)**.** *If neither feature value x nor threshold τ saturates and*

$$|x - \tau| > \frac{1}{Q},$$

then the truth value of $x > \tau$ equals that of $\mathcal{Q}_Q(x) > \mathcal{Q}_Q(\tau)$.

Proof. If $x > \tau + 1/Q$, Lemma 4.13 implies $\mathcal{Q}_Q(x)/Q \ge x - 1/(2Q) > \tau + 1/(2Q) \ge \mathcal{Q}_Q(\tau)/Q$. The reverse ordering is analogous. □

Corollary 4.16 (Argmax stability)**.** *Assume stable paths and unsaturated base/leaf quantization for v2. If the largest and second-largest ideal logits differ by more than $(R+1)/Q$, integer argmax returns the same class. Equal integer logits are resolved in favor of the lowest output index.*

Proof. Each of the two logits is perturbed by at most $(R + 1)/(2Q)$ by Theorem 4.14; the stated margin prevents reversal. The runtime replaces its current best index only on strict improvement. □

No unconditional rounding-only global error bound follows near a split. A small perturbation can select a different leaf, and the difference between alternate leaves need not be small. Accordingly, the stable-path hypothesis must accompany Theorem 4.14. For production binary64 preprocessing, each $1/(2Q)$ term in these results must be replaced by $(1/2 + |\delta_Q(z)|)/Q$, or the implementation must be shown to emit the same integer as the abstract quantizer for the values under analysis; these alternatives still require that the corresponding int32 clamp does not saturate. In particular, provided neither operand saturates, a sufficient production split margin is

$$|x - \tau| > \frac{1 + |\delta_Q(x)| + |\delta_Q(\tau)|}{Q}.$$

4.6 What registration alone proves

Proposition 4.17 (No unconditional registry content binding)**.** *For the deployed registry interface, the existence of a record under identifier h does not imply $h = \mathrm{keccak256}(B)$ for the bytes reconstructed from its table.*

Proof. Registration accepts h and the pointer metadata as independent arguments. It requires a nonzero unused h, but does not reconstruct and hash B. Choose any accepted pointer tuple reconstructing B and any nonzero unused $h \neq \mathrm{keccak256}(B)$. Subject to the remaining fees, metadata, and policy checks, this is an admissible counterexample. □

The executable isolated-chain test instantiates this counterexample against the unmodified contracts. The registry accepted `0x60c713...8875` for a tuple whose 104 reconstructed bytes hash to `0xeb0ee3...ab18`. The formal supplement records both complete 32-byte values. The runtime followed that record and returned the referenced bytes' prediction 1043. The production JavaScript chain loader now recomputes $\mathrm{keccak256}(B)$ after exact reconstruction and rejects the record when it differs from the requested identifier. This is a client-side mitigation, not a retroactive on-chain invariant.

Corollary 4.18 (Independent verification obligation)**.** *Theorem 4.11 applies to a deployed record only after a verifier establishes the canonical core definition, the manifest in Definitions 4.3 and 4.10. If the registry key is additionally claimed as a content identifier, that separate deployment-identity claim requires* $\mathrm{keccak256}(B) = \mathsf{modelId}$*; this equality is not a premise of execution equivalence for the reconstructed bytes.*

Proposition 4.19 (Independent local inference)**.** *An observer with ordinary EVM read access can reconstruct a well-formed GL1F model stored under the deployed `GL1C` scheme and evaluate it locally without using the canonical runtime endpoint.*

Proof. Pointer-table and chunk runtime code are public by address. Definition 4.3 constructively reassembles B. Equations 5 and 6 then require only public integer fields and a chosen packed input. □

5 Implementation

5.1 Binary layout and inference contract

All multibyte fields are little-endian. Canonical reserved fields are zero, and decoders reject unknown versions. Table 3 summarizes the two 24-byte fixed headers. In v2, the base vector of $4K$ bytes immediately follows the fixed header; trees follow in output-major order.

The input ABI is a byte string of exactly $4F$ bytes. Every feature is a little-endian int32 already expressed in Q-units. This placement is intentional: Theorem 4.11 starts at the packed vector and does not claim that arbitrary CSV parsers or application-specific preprocessing are

Table 3: Fixed GL1F header fields. Signed values are two's-complement int32.

Offset	Bytes	Type	v1 scalar	v2 vector
0	4	bytes	`GL1F`	`GL1F`
4	1	u8	version = 1	version = 2
5	1	u8	reserved = 0	reserved = 0
6	2	u16	feature count $F > 0$	feature count $F > 0$
8	2	u16	depth d	depth d
10	4	u32	tree count T	trees/output $R > 0$
14	4	i32	scalar base a_0	reserved = 0
18	4	u32	scale $Q > 0$	scale $Q > 0$
22	2	u16	reserved = 0	output count $K \geq 2$

equivalent. A scientific application should archive the preprocessing specification and the exact packed vector used for each reported prediction.

Forced leaves preserve the complete-tree layout when training cannot find a valid split. The remaining subtree is filled with feature zero, threshold $2^{31} - 1$, and duplicated leaf values. Traversal remains fixed depth, and Lemma 4.8 shows that the unused decisions are semantically inert for every packed int32 input, both because strict greater is unsatisfiable at the inserted threshold and because the descendant values are duplicated.

5.2 Training implementations

The browser, Python, and C++ trainers share the following structural choices:

- seeded `xorshift32` randomization;
- fixed-depth complete trees;
- histogram candidate generation with linear or quantile binning;
- deterministic feature, row, class, and label ordering under the publication profile;
- JavaScript-compatible finite-value rounding $\lfloor z+1/2 \rfloor$, followed by int32 saturation for serialized thresholds and leaves;
- scalar regression and binary logits in v1;
- output-major multiclass or multilabel logits in v2; and
- identical tree and header serialization rules.

Training is not integer-only. Split scoring, gradient updates, loss/metric evaluation, sigmoid/-softmax calculations, and early-stopping comparisons use floating point. Consequently, the paper makes two different reproducibility claims:

Inference determinism.

From valid fixed bytes and the same packed input, exact equality follows from Theorem 4.11.

Training parity.

For declared finite fixtures and execution conditions, tests compare the complete serialized core byte-for-byte. This is empirical evidence, not a theorem covering every CPU, compiler, transcendental library, malformed input, or reassociating optimization.

The publication profile requires identical finite matrices converted to float32, identical row/feature/output ordering, preprocessing completed before comparison, normalized task aliases and hyperparameters, the same canonical seed and random-consumption order, valid nonempty requested splits, bounded quantization without unexpected saturation, and an IEEE-754 environment that does not algebraically reassociate operations. The C++ build explicitly uses `-ffp-contract=off -fno-fast-math`. Core comparisons exclude the optional `GL1X` envelope, whose JSON metadata can contain non-semantic timestamps.

5.3 Corrections and hardening

The cross-runtime evaluation exposed a concrete IEEE-754 discrepancy in the browser's linear-bin threshold calculation. Python and C++ evaluated the normalized position using divide-then-multiply, whereas the browser worker used an algebraically equivalent multiply-then-divide expression. These expressions need not round to the same binary64 value. At an adversarial bin boundary, one threshold changed and therefore changed serialized model bytes. The worker now uses the same operation order as the native implementations, and the specific witness is retained as a production-path regression test.

The browser worker also now distinguishes an explicitly supplied seed of zero from an absent seed. This matters because zero is a valid user input even though the PRNG's internal canonicalization must avoid a permanently zero `xorshift32` state. Tests exercise repeatability and the seed-zero case separately.

An additional generated boundary case exposed a second floating-point discrepancy. In a generated 4,000-row, 12-feature, four-class profile, Python accumulated the multiclass softmax denominator from binary32-rounded exponentials, whereas JavaScript and C++ accumulated binary64 exponentials. The resulting 22,120-byte Python core differed from the other two engines at one byte, representing a one-Q-unit leaf difference. Python now uses the shared operation profile: binary64 exponentials, an ascending-class-index binary64 denominator sum, a binary32-rounded numerator, binary64 division, and a binary32 result. The production browser path also uses direct division rather than reciprocal multiplication. A generated three-engine regression locks the corrected complete-core SHA-256 to `58e3d9e63c5dd1313872e0c5a07c229b071272f1137f393a0384601329a432eb`. This closes the known counterexample; it is not a theorem about every compiler, architecture, or mathematical-library implementation.

The local JavaScript decoder was changed from a convenience parser into a strict inference boundary. It now enforces a depth ceiling, a core-size cap, positive feature count and scale, zero reserved fields, exact v1/v2 length, feature-index bounds, safe accumulator limits, and finite input features. These checks prevent a malformed public artifact from silently selecting an evaluator-dependent interpretation or requesting unreasonable allocation. The test oracle in `tests/publication/model_format.py` is an independent implementation rather than a call back into the production decoder.

The native trainer retains C++17 portability and uses standard stream time formatting in place of a fixed-size timestamp buffer. UI changes add visible focus states, reduced-motion behavior, forced-color support, responsive layout, print handling, and clearer distinctions among local execution, RPC-mediated EVM reads, and on-chain transactions. These accessibility and communication changes do not alter model semantics.

5.4 Solidity implementation and pinned compiler profile

The tested Solidity sources use pragma `^0.8.20`, but the tested deterministic profile is intentionally narrower: Solidity 0.8.20, optimizer enabled with 200 runs, `viaIR=true`, and `evmVersion=istanbul` [32]. Non-IR compilation fails with a stack-too-deep error. Sampled later IR compilers (0.8.22, 0.8.24, 0.8.26, 0.8.28, and 0.8.30) failed in Yul stack allocation for the current source. The repository therefore pins and checks the working compiler rather than treating the caret pragma as evidence that all permitted compilers produce a build.

All compiled runtimes fit below EIP-170 in this profile. The pinned live witness observed identical runtime lengths for the five deployed public addresses it queried. Runtime-length equality is useful consistency evidence, but is weaker than exact deployed-bytecode verification: compiler metadata, linked addresses, constructor effects, or a different byte sequence can share a length. Accordingly, this paper describes the examined source and the observed deployment without claiming a complete source-to-deployed-bytecode proof.

Table 4: Compiled runtime size under the pinned Solidity profile. The EIP-170 ceiling is 24,576 bytes.

Contract	Runtime bytes
`ForestRuntime`	17,794
`ModelRegistry`	12,725
`ModelNFT`	5,968
`ModelMarketplace`	3,828
`ModelStore`	714
`SimpleOwnable`	356

5.5 Deployed contract roles

`ModelStore` creates immutable payload code and rejects payloads above 24,572 bytes. `ForestRuntime` requires exact packed-feature length, decodes signed int32 values explicitly, uses int256 accumulators, and applies a lowest-index tie break for vector argmax. Its EIP-712 authorization path binds the model identifier, packed-input hash, deadline, chain identifier, and runtime address [4]. A view authorization is replayable until its deadline by design; it is not a one-time capability and provides no confidentiality.

`ModelRegistry` manages model settings, access plans, policy-version records, and the mapping to `ModelNFT`. The current NFT contract implements ownership, approvals, safe transfers, owner enumeration, burn, and custom raw metadata. It does not expose the full ERC-165/ERC-721 Metadata and Enumerable interface surface, so this paper calls it a model NFT contract rather than asserting universal wallet or marketplace interoperability. `ModelMarketplace` is approval-based and does not escrow the token while listed.

6 Evaluation

6.1 Research questions

The evaluation addresses five questions:

RQ1: Format and inference.
Do independent parsers and evaluators accept canonical v1/v2 fixtures, reject the enumerated malformed forms, and agree on boundary semantics?

RQ2: Training parity.
Under the finite publication profile, do the JavaScript, Python, and C++ trainers emit identical core bytes for the exercised tasks and controls?

RQ3: Reference performance.
What end-to-end wall time is observed for a fixed reproducible workload on one disclosed host?

RQ4: Contract integration and scaling.
Can unmodified contracts be compiled, deployed, populated through deliberately small chunks, and made to return the independent reference result for scalar and vector boundary cases? What scalar cost envelope is observed over six disclosed shapes?

RQ5: Live state.
At one number-pinned public chain state, can every active registered model be reconstructed and checked against its commitment, the nine relations in Definition 4.10, and nine deterministic local/provider-returned int32 call results?

Throughout the evaluation, "independent" means separately implemented or code-path separated from the component being checked. It does not mean independently authored, externally replicated, or produced by another laboratory.

6.2 Environment and artifacts

The frozen timing benchmark used Linux 6.18.35 x86-64 with glibc 2.39 on an AMD EPYC 9V74 host with nine logical CPUs visible, Python 3.12.13, NumPy 2.3.5[17], Node 24.19.0, and g++ 13.3.0. The older EVM-scaling and live-chain summary observations retain their recorded Node 24.14.0 provenance. The canonical parity matrix and isolated-EVM integration record Node 24.19.0, which is the release's exact `.nvmrc` pin. The release does not relabel earlier evidence after changing that rerun pin. The benchmarked trainer code is single-threaded; the host was neither CPU-pinned nor isolated. The C++ benchmark binary used `-O3 -DNDEBUG -std=c++17 -ffp-contract=off -fno-fast-math`; the publication test binary used `-O2 -ffp-contract=off -fno-fast-math -Wall -Wextra -pedantic`.

The public repository records source, locked JavaScript dependencies, Python requirements, locally reproducible tests and benchmark methods, strict parser code, expected local-fixture hashes, contract compiler settings, a local-EVM deployment test, and summarized live-chain results [13]. The raw provider responses, deployed model bytes, and collected runtime code underlying the historical live-chain summary are not included. The repository therefore supports direct reproduction of the local parity, formal-property, compilation, and isolated-EVM evidence; its live-chain numbers remain bounded, provider-attested historical observations.

6.3 Cross-runtime publication suite

The main parity dataset is generated without an external download. It contains 240 rows, five mixed-scale finite features, and deterministically constructed targets for regression, binary classification, three-class classification, and three-label classification. Baseline profiles train 18 depth-3 trees with learning rate 0.075, minimum leaf size four, 16 bins, scale $Q = 100{,}000$, 70/20/10 train/validation/test allocation, and seed 20,260,724.

Eighteen test methods passed. Their matrix covers:

- all four tasks with linear and quantile binning;
- automatic and manual class/label weighting, normalization, and stratification;
- configurations with early stopping enabled for every task and fixed-budget train+validation refitting;
- configurations with plateau and piecewise learning-rate schedules enabled;
- repeated-run determinism and explicit seed zero;
- reference-versus-production JavaScript inference;
- negative-half rounding, threshold equality, and strict-greater branching;
- v1/v2 lengths, GL1X framing, reserved bytes, version fields, feature indices, scale, dimensions, truncation, and trailing-data rejection; and
- the concrete linear-binning IEEE-754 operation-order witness and the generated 4,000-row/four-class softmax-precision witness.

These named cases are conformance probes, not exhaustive branch or path coverage; the study reports neither a coverage percentage nor a fault-detection rate. In the frozen artifacts, the enabled early-stop and plateau configurations do not realize an actual early termination or plateau drop; they test option propagation and continued-training parity, not the positive control-flow branches. Explicit seed-zero, repeatability, and production/reference v2-inference checks are auxiliary controls and are not additional profiles in the 25-profile training matrix.

Across the 25 distinct three-engine training profiles summarized by the suite, the JavaScript, Python, and C++ core SHA-256 digests were identical within every profile. The separately counted IEEE-754 operation-order case is an arithmetic witness, not a second execution of the duplicated regression training profile. Checked-in regression, binary, and multiclass Python/C++ artifacts also matched, with digests shown in table 5. The equality criterion was the entire core byte string, which is stronger than approximate prediction agreement on a sample but remains limited to the executed profile matrix.

Table 5: SHA-256 identities of checked-in Python/C++ artifact pairs.

Task	SHA-256
Regression	`a78f6918f0eca44d92e813a03ca4b3187d242164c38bdbdaa351fd47efc116aa`
Binary	`70c6b841266d58a063a6ecd54780726acfddf89cbb07a6e70a8019e4aeeeb84a`
Multiclass	`d530fe10d69e1e511d32ade2e15003c1f360812657aa4b0fe631fc5744104c87`

An additional eight executable property tests checked every root-to-leaf path against its leaf interval, the level-index invariant, selected boundary reads of lengths one, two, and four across nine chunk sizes, a concrete three-chunk failure when the chunk size is below four, pointer capacity, exact-rational quantization bounds, JavaScript accumulator bounds, and canonical equality of the checked-in native artifacts. All eight passed.

6.4 Reference trainer benchmark

The benchmark generated 3,000 rows with 12 deterministic numeric features and regression/binary targets. Each run used 60 depth-4 trees, learning rate 0.075, minimum leaf size eight, 32 linear bins, $Q = 100{,}000$, and seed 20,260,724. Each engine received one warm-up and 30 timed repetitions. Execution order rotated round-robin. The timer included process startup, input parsing, training, serialization, and output writing.

Python and C++ read the same CSV; JavaScript under Node read an equivalent JSON numeric matrix. The Node observation is therefore operational end-to-end time, not a pure language-kernel comparison. Table 6 reports the recorded medians and complete-core identity.

Table 6: Recorded end-to-end benchmark median (IQR width) in seconds; 30 timed repetitions per engine and task after one warm-up. Both task cores occupy 11,064 bytes.

Engine	Regression	Binary
Python	0.2847 (0.0183)	0.3074 (0.0146)
C++	0.0348 (0.0027)	0.0421 (0.0028)
Node JS	0.1355 (0.0105)	0.1503 (0.0082)

The ratios of recorded Python and C++ medians were 8.19 for regression and 7.30 for binary classification; Python-to-Node median ratios were 2.10 and 2.05. Regression cores shared SHA-256 `f354c657568232326d43c5a98a0b0ffa9392cc9bde7135b7d39c86f7f2d9f259`; binary cores shared `7d9db8594b84aa53d90d3d9abc5c6eef0d140c12ca95ddb768e0e1576b8829da`.

Thirty timed repetitions quantify within-host dispersion but do not establish a cross-host sampling distribution, confidence interval, or universal language speedup. Host load, serialization route, input parser, compiler, dataset size, depth, task, and CPU can change the ratios. No comparison with XGBoost, LightGBM, ONNX Runtime, or another on-chain system was performed.

6.5 Local EVM integration

The integration test compiles the unmodified contracts with the pinned Solidity profile and deploys `ModelStore`, `ModelRegistry`, `ModelNFT`, and `ForestRuntime` to an isolated Ganache 7.9.2 chain [7]. It wires the registry and NFT, then serializes:

- a 104-byte v1 fixture with two features, depth two, two trees, base 1,000, and $Q = 1{,}000$; and
- a 156-byte v2 fixture with two features, depth two, three outputs, one tree per output, base vector $(100, -50, 25)$, and $Q = 1{,}000$.

The test deliberately uses 17-byte payload chunks: seven chunks for v1 and ten for v2. This is not an efficient production setting. It forces two- and four-byte fields to cross logical chunk

boundaries and thereby exercises the two-chunk reader in Lemma 4.5. The test reads back every piece from runtime code, reconstructs the exact core, checks keccak256(B) = modelId, registers both models, and verifies registry/NFT wiring.

Six vectors exercise equality at negative, zero, and positive thresholds, strict-greater branches, both signed-int32 extremes, a three-way equal-logit tie, and an invalid feature-length revert. Independent bigint evaluation matched `predictView`, `predictMultiView`, and `predictClassView` for every valid vector. Transaction endpoints also emitted the independently expected scalar or vector.

The same integration run exercised the assurance boundary in Proposition 4.17: it registered the valid v1 storage tuple under a distinct nonzero identifier, confirmed that `predictView` still reached those bytes and returned 1043, and confirmed that the production chain loader rejected the registered/reconstructed hash mismatch. This turns the source-level counterexample into an executable regression test for the required client-side check.

In the recorded tiny-fixture run, mean estimated view gas was 77,600 for v1, 109,011 for v2 vector output, and 108,412 for v2 class output. Mined inference transactions used 79,573 gas for v1 and 115,527 gas for v2 vector output. These figures are regression witnesses for the exact fixtures, compiler, and Ganache configuration. They are not a formula for large public models, a network fee estimate, or evidence that every encoded model is affordable.

6.6 Local-EVM scalar scaling

A supplementary benchmark uses the same Solidity 0.8.20, IR, optimizer, Istanbul-target, and Ganache/Shanghai execution profile, while replacing the 17-byte correctness chunks with 20,000-byte payload chunks. It serializes six deterministic two-feature v1 model shapes, publishes and reconstructs each core, and evaluates 12 seeded int32 feature vectors per shape against the independent bigint interpreter. All 72 comparisons matched.

An ordinary least-squares description of these six mean observations against tree-body model-code read count R_{tree} is

$$\widehat{\text{gas}} \approx 42{,}900 + 1{,}708.3R_{\text{tree}},$$

with $R^2 \approx 0.99998$ and maximum absolute relative residual 1.45%. The fit uses unrounded arithmetic means. The close fit is descriptive evidence for this synthetic shape family, not a gas theorem: compiler, client, hardfork, calldata, chunk geometry, selected chunk addresses, and address warmth can change the coefficients and input-level gas. The exact decision/leaf-read counts remain portable; the fitted coefficients do not.

Table 7: Local scalar v1 scaling observations. Gas values are arithmetic means of 12 `eth_estimateGas` calls over seeded vectors, rounded to the nearest gas unit; they are not mined receipts or live-network fee forecasts. Tree-body model-code reads equal $T(2d+1)$ and exclude registry, header, and pointer-resolution work.

T, d	Core bytes	Chunks	Decisions / reads	Mean gas
20,3	1,784	1	60 / 140	286,247
50,3	4,424	1	150 / 350	644,454
50,4	9,224	1	200 / 450	810,777
100,4	18,424	1	400 / 900	1,576,945
200,4	36,824	2	800 / 1,800	3,121,210
50,6	38,024	2	300 / 650	1,146,414

6.7 Pinned GenesisL1 deployment witness

The live-state study queried GenesisL1 chain ID 29 at block 13,342,043, whose timestamp is 24 July 2026 17:23:05 UTC and whose recorded hash is

`0xffd825db1bb2534052a604db9584d361111d8bc9e19d753b0ee3861bf320d1b9`.

Every getter, `eth_getCode`, `eth_call`, and historical `eth_estimateGas` request used that numeric block tag. No transaction or raw transaction was sent. After all calls, the witness fetched the block again and required the recorded hash to be unchanged. The extended record identifies `https://rpc.genesisl1.org` as the selected endpoint; repeating the witness through independently operated archival providers would further strengthen the observation. The raw provider responses and reconstructed model and runtime bytes are not included in the public repository.

The observed public addresses were:

Table 8: Contracts queried by the pinned witness.

Role	Address
Store	`0x9CdbC23392648Bd27B4A5eD09c0fEa9452454B54`
Registry	`0x33c9844F77a07e36B98f0FFf8201B8A8b02c2a69`
Model NFT	`0x44Dc1c54B8D579B42d78cC21cf8260DC0A3279fA`
Runtime	`0xD2fD0cf461a6cb56Fc08d9aEc120833D8E79044E`
Marketplace	`0xA9bfa0a719b7F73cE85CA1E7f23af626D383fB46`

For each active token, the script read an exact pointer table; required every 32-byte address slot's high 12 bytes to be zero; required $N = \lceil \ell/c \rceil$; checked every prefix and exact chunk payload length; reconstructed exactly ℓ bytes; strictly decoded the header and every feature index; checked the computed core length and the nine relations in Definition 4.10—$nFeatures$, total $nTrees$, $depth$, $baseQ$ (the v1 scalar base or the v2 zero-reserved convention), $scaleQ$, $tablePtr$, $totalBytes$, $numChunks$, and $chunkSize$; compared keccak256(B) with the registered identifier; and executed nine deterministic packed int32 vectors. For every model those vectors comprised all-zero, all-INT32_MIN, all-INT32_MAX, and two complete $\{\theta-1, \theta, \theta+1\}$ triplets around distinct always-visited root thresholds. A separate direct-int32 evaluator computed the local outputs and reported source-instrumented model-code reads before comparison with historical-state `eth_call` simulation outputs.

These deterministic vectors are conformance probes, not exhaustive path coverage. The root-neighborhood cases exercise their named root decisions but do not visit every internal node or leaf; no path-coverage percentage or fault-detection rate is claimed.

Table 9: Aggregate result at the pinned deployment state.

Quantity	Observation
Active model records	12
Distinct creator addresses	2
Reconstructed core bytes	31,185,324
Commitments equal to keccak256(B)	12/12
All nine registry/core/storage relations	12/12 models
Exact local/provider-returned call matches	108/108
Historical gas estimates returned	108/108
Source-instrumented model-code reads / crossings	1,604,970 / 17
Nominal nonfinal payload/stride c	24,000 bytes (all models)
Individual model-size range	816–9,523,096 bytes
Chunk-count range	1–397

Table 10 gives the per-record storage geometry. The largest observed object was a 9,523,096-byte depth-9 ensemble with 1,552 trees, reconstructed from 397 chunks. The recorded provider-attested summary indicates that the deployed storage and read path handled a model far larger than one EIP-170 contract, while remaining below the one-table capacity bound.

Table 10: Per-model records reconstructed at block 13,342,043. "Trees" is total serialized trees and $c = 24{,}000$ bytes for every row. Every row passed the commitment, all nine named relations, and all nine deterministic local/provider-returned call checks described above. Titles are unverified registry metadata.

Token	Title	Ver.	F	d	Trees	Bytes / chunks
1	Iris model	2	4	3	1,800	158,436 / 7
2	Raisin classification	1	7	5	169	63,568 / 3
3	Boston housing prices	1	13	6	196	148,984 / 7
4	PV efficiency, 57° North	1	4	3	275	24,224 / 2
5	Bitcoin 24 h volatility	1	53	5	20	7,544 / 1
6	ETH +2% in 5 h	1	26	8	1,496	4,583,768 / 191
7	ETH −2% in 5 h	1	27	8	2,500	7,660,024 / 320
8	XRP +2% in 5 h	1	33	8	1,008	3,088,536 / 129
9	XRP −2% in 5 h	1	33	9	1,552	9,523,096 / 397
10	ZEC +4% in 5 h	1	33	7	1,789	2,733,616 / 114
11	ZEC −4% in 5 h	1	33	8	1,042	3,192,712 / 134
12	ZEC ±4% gate	1	47	3	9	816 / 1

Table 11: Historical `eth_estimateGas` ranges over the nine deterministic vectors for each record. These are node simulations at the pinned block, not mined transaction receipts.

Token	Minimum	Maximum	Token	Minimum	Maximum
1	22,782,413	22,784,217	7	86,163,819	86,178,402
2	3,213,782	3,213,872	8	31,373,837	31,379,108
3	4,397,999	4,398,166	9	57,181,535	57,190,348
4	3,355,295	3,355,955	10	50,986,502	50,993,025
5	419,789	422,265	11	32,514,162	32,515,585
6	48,187,258	48,195,195	12	155,671	157,902

The maximum returned estimate was 86,178,402 gas for token 7. The pinned block's recorded gas limit was 1,000,000,000, making the estimate 8.62% of that limit. This is a comparison of one node's historical simulation with one block header, not evidence of a mined transaction, latency, affordability, or portable execution cost. The 17 instrumented cross-chunk reads all occurred in the v2 record (token 1); for the archived v1 paths, two- and four-byte field alignment with the 24,000-byte stride produced no crossings.

The two reserved bytes in each eight-byte internal-node record have no inference semantics in the current format and must be zero. Across this pinned 31,185,324-byte corpus they occupy exactly 5,189,600 bytes, or 16.64% of all core bytes. Token 9 alone contains 1,586,144 such bytes. Counterfactually, retaining all other fields while using six-byte nodes would reduce that model's core from 9,523,096 to 7,936,952 bytes and, at $c = 24{,}000$, from 397 to 331 chunks. This is a quantified v3 design opportunity, not a change that can be applied to existing v1/v2 bytes or already deployed runtime code.

The "Boston housing prices" string for token 3 is only registry metadata observed at the pinned block: the witness did not recover a dataset citation or training provenance and therefore cannot establish that the model used the well-known Boston Housing dataset. If that title does refer to `sklearn.datasets.load_boston`, its maintainers deprecated and removed the loader because of documented ethical concerns and strongly discouraged ordinary use [30]. Any scientific reuse must resolve provenance, justify the dataset and feature construction, and perform application-specific ethics review; the title and on-chain persistence do not provide that justification.

The two creator addresses are not evidence of two independent people or organizations. Registry titles are not scientific validation. "Independent" here describes separately implemented parsing or evaluation logic, not an independent author or laboratory. Two named public RPC

routes supplied the historical observations, but the study does not establish that their node implementations or infrastructure are independent. An `eth_call` response is not a transaction receipt or a consensus proof. The recorded summary reports code-hash checks against collected runtimes, but neither those runtimes nor the raw responses and proof paths are included in the public repository. Consequently, the historical observation cannot be replayed offline from this release; its method and stated results can be tested only by a fresh provider-assisted rerun against retained historical state.

6.8 What the evaluation does not establish

None of the preceding results establishes:

- predictive accuracy, calibration, robustness, causal meaning, external validity, fairness, or fitness for any named scientific, clinical, financial, or governance use;
- absence of data leakage, sampling bias, distribution shift, label error, adversarial fragility, or unsafe feature construction;
- authorship, dataset licensing, intellectual-property ownership, or legal enforceability of registry metadata;
- privacy of weights, inputs, outputs, signatures, or access patterns;
- permanent availability of a mutable inference service, RPC endpoint, marketplace listing, access plan, or token record;
- security of GenesisL1 consensus, wallets, bridges, RPC providers, or user devices; or
- cost superiority over ordinary off-chain serving, proof-based verification, or another chain.

An application-specific scientific study must add a named and versioned dataset, inclusion/exclusion criteria, frozen splits, baselines, prespecified metrics, uncertainty estimates, leakage analysis, calibration where relevant, subgroup analysis, ablations, and domain-specific ethical and safety review.

7 Trust and security boundaries

7.1 Threat model

The primary integrity objective is that an independent observer can determine which public bytes were evaluated and reproduce the exact integer result. The analysis considers:

- a publisher who supplies malformed bytes, false metadata, an unrelated identifier, or an inconsistent storage shape;
- an observer who attempts to claim a visible content identifier before its intended publisher registers it;
- a mutable NFT owner, fee recipient, listing, access plan, or administrator setting;
- an RPC provider that returns incomplete or inconsistent historical state;
- malformed or non-finite browser input; and
- compiler drift that changes or prevents deterministic compilation under the pinned profile.

The paper does not model a failure of the underlying chain's consensus, a compromised wallet or browser, key theft, validator collusion, or a malicious compiler binary. Those remain external assumptions. Multiple independent RPC providers, block-header verification, independently rebuilt toolchain artifacts, and deployed-bytecode verification can strengthen those layers but are not silently assumed here.

7.2 Positive properties of the deployed design

Several properties reduce the execution surface:

- payload chunks contain immutable runtime code and no mutable `SSTORE`-backed model state;
- the store enforces the code-size-derived payload limit and detects a failed `CREATE`;

- the runtime requires exact packed-feature length and explicitly decodes signed little-endian int32 values;
- complete trees eliminate data-dependent loop counts within one tree;
- int256 accumulation avoids int32 sum wraparound;
- strict-greater traversal and lowest-index argmax ties are deterministic;
- fee forwarding and inference occur atomically in transaction paths;
- typed owner signatures are domain-separated by chain and runtime and reject invalid v and high-s ECDSA values;
- model identifiers are single-use; and
- NFT transfer authorization, approval clearing, nonzero-recipient checks, receiver callbacks, and owner-array maintenance are implemented.

These properties do not remove the need for strict model verification. In particular, a four-byte magic value detects many mistakes but does not prove content identity.

7.3 Current deployed limitations

The following limitations describe the deployed contract interface and are material to the interpretation of this paper.

Caller-supplied content identity. As proved in Proposition 4.17, the registry does not compute the model hash. Chunk deployment is visible before the later registration transaction, so a canonical identifier can also be preemptively registered by another account. Because burned identifiers remain used, denial of the canonical record can be permanent. The separately implemented witness closes this evidence gap for the 12 observed records only; it does not change the registry predicate for future records.

Task identity outside the core commitment. The same content identifier can denote v1 bytes later described as either a regression score or binary logit, or v2 bytes described as either multiclass or multilabel logits. Because no task discriminator or preprocessing manifest is included in B, a correct $\mathsf{modelId} = \mathrm{keccak256}(B)$ authenticates neither interpretation. Existing records therefore need a separately authenticated semantic manifest; a successor format can domain-separate task and semantic commitments without pretending to alter deployed v1/v2 bytes.

Partial format validation. Registration checks positive chunk count and size but does not establish exact table/chunk length, table capacity, exact core geometry, all magic/version fields, feature-index validity, or full header/registry equality. The scalar runtime trusts registry fields and begins tree reads at offset 24 rather than strictly decoding the complete v1 header. A future malformed registration can therefore revert, evade declared-size economics, or be interpreted differently by a permissive local client.

Transfer and access semantics. NFT transfer does not atomically rotate the permanent owner access key or change the fee recipient. A previous owner can retain a key until it is revoked, while fees can continue to a prior/custom recipient until settings are updated. This may be an intentional separation of record ownership from service rights, but it must be shown to a buyer rather than inferred from the token transfer.

Mutable service layer. The current token owner can disable inference, change pricing, fee destination and plans, revoke keys, or burn the record. The administrator can replace the NFT contract used for ownership checks and can emergency-burn. Stored chunk bytes remain available

by address, but the canonical service and discovery record need not remain active. Immutability of bytes is therefore not immutability of access or marketplace economics.

Pricing and listings. Pricing mode is stored as an unconstrained uint8 although the UI documents three modes. Values above two can be displayed as paid yet remain callable through some public view paths. Marketplace listings are approval-based: direct token transfer can leave a stale listing that becomes executable again if the token returns to the original seller and approval survives. External call ordering can also produce a confusing sequence of listing events even when final state is safe. Stale-state and economic-semantics risks therefore remain relevant to applications using this optional layer.

Standards and metadata. The NFT implements a useful ownership subset but not ERC-165 `supportsInterface`, standard `tokenURI`, or the full Enumerable interface. Task names and feature labels are custom metadata and are not reconciled on-chain with the core's version, output count, or feature count. A UI that selects an endpoint from an inconsistent task string can call the wrong function. Scientific clients must derive execution shape from the strictly decoded core and treat names as labels.

7.4 Hardening that works with the existing deployment

Several protections require no change to deployed bytecode:

1. make strict reconstruction and verification mandatory before a client displays a model as valid;
2. show the computed identifier beside the registered identifier and refuse inference in the canonical UI when they differ; the production JavaScript loader now performs this reconstruction-time hash check;
3. derive version, feature count, output count, depth, and expected length from the validated core, then reconcile rather than trust free-form metadata;
4. reject missing or non-finite feature input, state units and allowed ranges, and expose the exact packed int32 vector in an exportable receipt;
5. display public-download status, current fee recipient, surviving access keys, pricing mode, inference-enabled state, owner, and administrator powers before subscription or purchase;
6. pin a block number and hash for scientific reports, archive all chunk addresses and code hashes, and query more than one historical RPC where possible; and
7. keep the compiler version and settings locked in CI and archive source, creation bytecode, deployed runtime bytecode, and their hashes.

These measures can make claims about an individual existing record robust. They do not retroactively add an on-chain invariant.

7.5 Successor-contract migration

A stronger registry requires new bytecode and an explicit migration; it must not be described as already deployed. A backwards-compatible successor design should:

1. add a publisher-bound reservation or commit/reveal phase before public chunk upload, preventing third-party capture of an intended identifier;
2. authenticate a domain-separated manifest containing exact total length, chunk size, ordered chunk commitments or code hashes, core hash, and canonical header fields, with a separately versioned task discriminator and semantic/preprocessing-manifest commitment;
3. enforce a shared strict v1/v2 validator for every endpoint, including exact length, dimensions, scale, version, reserved fields, and registry/header agreement;
4. constrain pricing with an enum and emit events for every material settings change;

5. define token-transfer semantics explicitly and, if service control is meant to follow ownership, atomically rotate the owner key and confirm the fee destination while leaving subscriber keys distinct;
6. add permissionless stale-list cleanup, checks-effects-interactions, explicit relisting, and reentrancy protection to the marketplace;
7. use a standards-tested ERC-721 implementation or clearly version the current minimal interface; and
8. make the ownership-contract reference immutable or use a timelocked, manifest-driven migration with explicit token mapping.

Two content-binding strategies are available. A registry can compute keccak256(B) by streaming all bytes during registration, which is simple but duplicates potentially large read cost. Alternatively, it can bind a Merkleized ordered manifest whose leaves commit to exact chunk payloads and lengths, while an independent verifier checks the full core. The latter reduces registration work but shifts the security statement: the registry authenticates the manifest root, and exact core identity additionally depends on the specified leaf construction. The migration specification should state this difference explicitly and include adversarial conformance vectors.

8 Limitations and validity

Finite parity matrix. The three trainers were compared across a broad but finite fixture matrix. The suite is designed to catch differences in task structure, binning, weighting, seeding, schedules, and stopping, but it is not an exhaustive state-space proof. Floating-point training can diverge on another compiler, library, architecture, dataset, tie, overflow, or optimization mode.

Synthetic parity and timing data. The primary cross-engine suite and performance workload are deterministic synthetic datasets chosen for reproducibility and boundary probing. They are not scientific application benchmarks and yield no evidence about real-world accuracy. The timing study has 30 measurements per engine and task on one non-isolated host, so it quantifies within-host dispersion rather than a cross-host performance population.

Gas evidence. Formal work counts are independent of gas schedules, but empirical gas depends on compiler, EVM revision, calldata, chunk geometry, model shape, cold/warm access, and network configuration. The correctness fixtures are intentionally tiny; the supplementary scalar sweep reaches only 38,024 bytes, two features, v1, and one synthetic family. The live witness obtained 108 historical `eth_estimateGas` responses from its selected route, but they remain node simulations rather than mined receipts or a transferable fee model.

One live state and two public RPC routes. The recorded witness summary is pinned to one historical GenesisL1 state. It reports that two routes returned the same block hash, per-model lengths, content commitments, and 108 decoded outputs. The raw provider responses and reconstructed deployed bytes are not included, so the historical observation cannot be replayed offline from this release. The observations remain provider-attested: the experiment does not prove independent infrastructure, validate account-proof paths against the state root, or authenticate the header chain. It is not evidence about later registrations, reorganizations outside the guard used, independent chain adoption, or another EVM network.

Finite live inputs. The extended witness evaluated nine deterministic vectors per model: all-zero, both signed extrema, and two complete root-threshold neighborhoods. The resulting 108 matches exercise exact equality, one-unit neighbors, and extreme packed inputs, but they are not exhaustive over the F-dimensional int32 domains, do not visit every internal node or

leaf, are not samples from a target population, and do not estimate predictive performance. No path-coverage percentage or fault-detection rate is reported.

Developer-conducted evaluation. The protocol's developer is the sole author and conducted the reported experiments. Source-separated oracles and machine-readable records reduce shared-code failure, but they are not independent replication. Re-execution by an unaffiliated group, independently operated infrastructure, and verified proof paths would increase confidence.

Source/deployment identity. Runtime byte lengths matched the pinned compiler output, but the study did not produce a full source-verification proof for each deployed address. Formal claims apply to the examined source and to deployed behavior only where the live witness directly exercises and checks the relevant predicate.

Privacy and public economics. Weights are public by design. Feature vectors passed in transactions become public calldata. A read-only `eth_call` does not itself place its input on-chain, but discloses it to the selected RPC provider, which may log it; signed view requests provide authorization, not protocol-level secrecy. Users needing confidential weights or inputs should select a different architecture, such as trusted execution, multiparty computation, or proof-based evaluation, and analyze its separate trust assumptions.

9 Reproducibility

Original GL1F software is released under the MIT License except where a file or directory states another license. Third-party and separately licensed software retains its upstream terms as recorded in `THIRD_PARTY_NOTICES.md`.[13] A complete local verification run installs locked JavaScript dependencies and executes:

```
npm ci
npm run verify
```

The aggregate command runs the 18-test publication suite, eight formal-property tests, browser static checks, the pinned Solidity compilation, and local EVM integration. Individual commands are:

```
python3 -m unittest -v tests.publication.test_publication
python3 -m unittest -v \
  tests.contracts_publication.test_formal_properties
python3 tests/ui_static_check.py
node tests/contracts_publication/compile_contracts.mjs
node tests/contracts_publication/test_evm_integration.mjs
```

The timing record is regenerated with:

```
python3 benchmarks/publication_benchmark.py \
  --rows 3000 --features 12 --trees 60 --repeats 30 \
  --out benchmarks/results/publication_benchmark.json
```

The local scalar EVM scaling record is regenerated with:

```
npm run benchmark:evm
```

The historical live-state values are retained as a provider-attested summary. The public repository includes the read-only collection method but not the raw provider responses or reconstructed deployed bytes, so it does not provide an offline reproduction of that observation. The local parity, formal-property, pinned compilation, benchmark, and isolated-EVM commands above remain reproducible from the public source.

10 Discussion

The strongest property of GL1F is architectural rather than promotional. A fixed-width complete-tree representation reduces a prediction to a sequence of specified byte reads, signed integer comparisons, heap-index updates, and integer sums. Each premise can be independently checked. A verifier can report the first violated invariant instead of asking a user to trust a remote model service or an unqualified statement that an asset is "on-chain."

This design also exposes a useful three-layer model of reproducibility:

1. **computational identity**: the same valid bytes and packed input define the same integer output;
2. **deployment identity**: a pinned manifest actually reconstructs those bytes, and its content commitment matches; and
3. **scientific validity**: the dataset, experimental design, model behavior, uncertainty, and intended use support a domain claim.

GL1F provides mechanisms and evidence for the first two layers. The third cannot be inherited from a hash, token, transaction, or consensus result. This separation is a strength: it prevents ledger provenance from being misrepresented as epistemic authority while still making a meaningful part of the computational record durable and contestable.

Complete fixed-depth trees trade storage for regularity. A sparse tree encoding would often be smaller, but would complicate offsets, introduce variable path or node structures, and widen parser and proof surfaces. GL1F's forced-leaf convention preserves regularity at the cost of duplicated data. Future work could investigate authenticated sparse layouts, compressed threshold tables, or batched chunk reads while maintaining a formally simple interpreter. A less disruptive v3 option is a six-byte internal-node record that removes the currently zero two-byte reserve. The live-corpus calculation above shows a 16.64% corpus saving and a 397-to-331 chunk reduction for the largest observed model, but adopting it would require a new version, decoder branch, proof, and migration path.

The live witness reconstructed a 9.52 MB model and 31,185,324 bytes across the pinned registry state. It also demonstrates why independent verification must remain first-class. All 12 records happened to satisfy the checked binding, but the current registry does not enforce it. Positive observations support the specific state; they do not erase the counterexample in Proposition 4.17.

11 Conclusion

GL1F provides a public, reconstructible execution artifact for fixed-depth integer GBDTs across browser, Python, C++17, and the EVM. Its scalar and vector formats have exact geometry; traversal and work are statically bounded; and, under explicit same-state manifest, metadata, input, and arithmetic premises, conforming local and EVM evaluator specifications are exactly equivalent. Source inspection, compiled tests, and pinned observations separately support the tested implementation. Fixed-point approximation admits useful conditional error and margin bounds, while the failure of a global rounding-only bound near splits is stated rather than hidden.

The implementation evidence comprises byte-identical cores across 25 distinct three-engine training profiles plus one standalone IEEE-754 arithmetic witness, strict malformed-format tests, independent formal-property witnesses, recorded single-host reference timings, a local contract deployment with chunk-boundary cases, 72/72 scalar reference/EVM comparisons over six measured shapes, and a provider-attested pinned state for which the recorded provider summary reports 12 reconstructed models, 108/108 matched deterministic read calls, and 108 returned historical gas estimates. The raw provider responses and reconstructed deployed bytes are not included, so those figures remain a non-independent historical observation. The executable mismatched-identifier registration and strict-loader rejection also make the deployed registry's

assurance boundary directly testable. The observed deployment includes models from 816 bytes to 9.52 MB and shows that immutable code chunks can support ensembles well beyond one contract's runtime limit.

The result is not that a blockchain makes machine learning true. It is that a specific public model computation can be made inspectable, content-addressed, and deterministically reevaluated by conforming, separately implemented evaluators. That is a bounded but scientifically valuable primitive for reproducible computational systems.

A Canonical conformance checklist

An implementation claiming compatibility with the publication profile should produce an explicit report for each item below.

A.1 Core

1. The first four bytes equal `GL1F`.
2. Version is exactly one or two; unknown versions are rejected.
3. Fixed header length is available before any field read.
4. Reserved bytes and fields are zero.
5. $F > 0$, $Q > 0$, and first-party depth satisfies $1 \le d \le 12$.
6. For v1, T is decoded as u32; publication through the deployed registry separately requires $T \le 2^{16} - 1$.
7. The application records whether v1 is regression/binary or v2 is multiclass/multilabel and authenticates that semantic manifest separately, because the core hash contains no task discriminator.
8. For v2, $K \ge 2$, $R \ge 1$, and trees are output-major.
9. Checked arithmetic computes $b(d)$ and exact core length before allocating or traversing.
10. The supplied byte string has exactly the computed length, without truncation or inference-relevant trailing bytes.
11. Every feature index is less than F.

A.2 Storage manifest

1. $4 \le c \le 24572$, where c is the nominal nonfinal payload and offset stride; $N = \lceil \ell / c \rceil$, and pointer capacity is sufficient.
2. The table has exact `GL1C` framing and exactly N slots.
3. Every address is nonzero and every chunk begins with `GL1C`.
4. All nonfinal payloads have exactly c bytes; the final payload has exactly $\ell - c(N-1)$ bytes.
5. Concatenation reconstructs exactly ℓ declared bytes.
6. All nine relations in Definition 4.10 hold.
7. keccak256(B) equals the registered identifier.
8. The verification record identifies table/chunk addresses and code hashes at a pinned block number and hash, and states whether its raw provider responses are available.

A.3 Inference

1. Packed input length is exactly $4F$ and every element is decoded as little-endian int32.
2. Application preprocessing rejects non-finite values and records saturation or imputation.
3. Equality to a threshold traverses left; only strict greater traverses right.
4. Accumulators preserve exact mathematical integer sums.
5. Vector argmax replaces the current class only on strict improvement, giving the lowest-index tie rule.

6. Test vectors include equality, one-unit neighbors, signed extremes, negative-half quantization, and equal logits.

B Proof-obligation matrix

Table 12: Premises required by each principal formal result.

Result	Required premises	Does not establish
Tree byte length	Complete depth-d tree; 8-byte internal nodes; 4-byte leaves	Compression efficiency, affordability, or training quality
Exact reconstruction	Exact table/chunk framing, content, order and lengths; $4 \le c \le 24572$; in-range 1/2/4-byte read	Authentication from magic alone or honesty of a registry record
Leaf safety	Canonical complete-tree geometry; exactly d strict decisions	Safety of malformed offsets or arbitrary depth allocation
Finite work	Validated finite T, d, and K, R for v2	Execution within a particular gas limit
Local/EVM equality	Canonical core; well-formed manifest; all nine named registry/-core/storage relations; identical packed int32 input; same strict predicate; exact sums	Equivalent preprocessing, training, metadata, accuracy, or chain truth
JavaScript exact sum	Deployed registry count bound and one-table storage capacity	Safety for arbitrary external files or a future enlarged format
Single-value error	Finite real value, positive scale, exact-real product, no int32 saturation	Path preservation or ensemble error by itself
Stable-path error	No changed path; abstract exact-real base/leaf quantization; no saturation	A global bound near thresholds
Split stability	Abstract exact-real quantization, no saturation, and real margin $> 1/Q$ at the node; production uses the δ_Q-corrected margin	Necessity of the condition or whole-tree stability without checking each visited split
Argmax stability	Abstract exact-real quantization; stable paths; no saturation; top-two ideal-logit margin $> (R + 1)/Q$	Calibration, probability accuracy, or classification validity

C Application-level scientific record

For a GL1F model to support a scientific result rather than only a artifact-to-execution claim, its accompanying record should include:

1. a persistent dataset identifier, licence, collection context, inclusion/exclusion criteria, and exact frozen data digest;
2. feature names, units, transformations, missingness policy, allowed range, and order matching F;
3. target definition, temporal cutoff, class/label order, and prevention of subject, temporal, or group leakage;
4. immutable train/validation/test membership or a fully specified split generator and seed;
5. model-selection protocol, all evaluated hyperparameters, stopping criterion, and treatment of validation data during any refit;
6. baselines chosen before test evaluation, task-appropriate metrics, confidence intervals or repeated resampling, and calibration analysis where scores are interpreted probabilistically;
7. ablations separating the effect of model family, feature set, quantization scale, depth, tree count, and deployment conversion;
8. subgroup, shift, failure-case, sensitivity, and external-validation analyses appropriate to the domain;
9. the GL1F core, GL1X or external metadata, trainer revision, environment, strict parser report, source and artifact hashes;
10. the exact storage manifest, block number/hash, packed conformance inputs, local integer outputs, and provider-returned call outputs; and
11. a plain-language intended-use and non-use statement identifying who can be affected and which human review remains necessary.

This record lets readers locate disagreements. A failed hash is a deployment identity problem; a mismatched integer output is an evaluator problem; a different packed vector is a preprocessing problem; poor external performance is a scientific-validity problem. Conflating these layers prevents effective review.

References

[1] 0xSequence. SSTORE2: Contract code as write-once data storage. Open-source Solidity library, n.d. URL https://github.com/0xsequence/sstore2. Repository accessed 27 August 2026; no source revision is implied by this conceptual citation.

[2] Abdulrahman Alhaidari, Balaji Palanisamy, and Prashant Krishnamurthy. On-chain decentralized learning and cost-effective inference for DeFi attack mitigation. In *7th Conference on Advances in Financial Technologies*, volume 354 of *Leibniz International Proceedings in Informatics*, pages 35:1–35:27. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025. doi: 10.4230/LIPIcs.AFT.2025.35.

[3] Syed Badruddoja, Ram Dantu, Yanyan He, Mark A. Thompson, Abiola Salau, and Kritagya Upadhyay. Making smart contracts predict and scale. In *2022 Fourth International Conference on Blockchain Computing and Applications (BCCA)*, pages 127–134. IEEE, 2022. doi: 10.1109/BCCA55292.2022.9922480.

[4] Remco Bloemen, Leonid Logvinov, and Jacob Evans. EIP-712: Typed structured data hashing and signing. Ethereum Improvement Proposals, 2017. URL https://eips.ethereum.org/EIPS/eip-712.

[5] Vitalik Buterin. EIP-170: Contract code size limit. Ethereum Improvement Proposals, 2016. URL https://eips.ethereum.org/EIPS/eip-170.

[6] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data*

Mining, pages 785–794. Association for Computing Machinery, 2016. doi: 10.1145/2939672.2939785.

[7] ConsenSys Software Inc. Ganache, version 7.9.2. Local Ethereum development blockchain software, n.d. URL https://github.com/trufflesuite/ganache. Archived upstream repository and package documentation; accessed 27 August 2026.

[8] K. D. Conway, Cathie So, Xiaohang Yu, and Kartin Wong. opML: Optimistic machine learning on blockchain. arXiv preprint, 2024. URL https://arxiv.org/abs/2401.17555.

[9] George Danezis, Markulf Kohlweiss, Benjamin Livshits, and Alfredo Rial. Private client-side profiling with random forests and hidden markov models. In *Privacy Enhancing Technologies*, volume 7384 of *Lecture Notes in Computer Science*, pages 18–37. Springer, 2012. doi: 10.1007/978-3-642-31680-7_2.

[10] Decentralized Science Labs. GenesisL1 Forest (GL1F): A scientific explanation and technical documentation for an on-chain EVM gradient-boosted tree studio. Technical report, Decentralized Science Labs, January 2026. URL https://github.com/GenesisL1/Forest/blob/722c0513d8da2a241de2cabaad1bd8c27791b7e7/GL1F.pdf. Public protocol document dated 19 January 2026.

[11] Vaidotas Drungilas, Evaldas Vaičiukynas, Mantas Jurgelaitis, Rita Butkienė, and Lina Čeponienė. Towards blockchain-based federated machine learning: Smart contract for model inference. *Applied Sciences*, 11(3):1010, 2021. doi: 10.3390/app11031010.

[12] William Entriken, Dieter Shirley, Jacob Evans, and Nastassia Sachs. EIP-721: Non-fungible token standard. Ethereum Improvement Proposals, 2018. URL https://eips.ethereum.org/EIPS/eip-721.

[13] Mikhail Fedorov. GenesisL1 Forest (GL1F): Reference implementation. Software, version 0.2.3, 2026. URL https://github.com/GenesisL1/Forest. Version 0.2.3, 5 September 2026.

[14] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

[15] Moxuan Fu, Chuan Zhang, Chenfei Hu, Tong Wu, Jinyang Dong, and Liehuang Zhu. Achieving verifiable decision tree prediction on hybrid blockchains. *Entropy*, 25(7):1058, 2023. doi: 10.3390/e25071058.

[16] Andrew Gan and Zahra Ghodsi. Sentry: Authenticating machine learning artifacts on the fly. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS '25, pages 3550–3563. Association for Computing Machinery, 2025. ISBN 979-8-4007-1525-9. doi: 10.1145/3719027.3765070.

[17] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[18] Justin D. Harris and Bo Waggoner. Decentralized and collaborative AI on blockchain. In *2019 IEEE International Conference on Blockchain (Blockchain)*, pages 368–375. IEEE, 2019. doi: 10.1109/BLOCKCHAIN.2019.00057.

[19] Everett Hildenbrandt, Manasvi Saxena, Nishant Rodrigues, Xiaoran Zhu, Philip Daian, Dwight Guth, Brandon Moore, Daejun Park, Yi Zhang, Andrei Stefanescu, and Grigore Roşu. KEVM: A complete formal semantics of the Ethereum virtual machine. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*, pages 204–217. IEEE, 2018. doi: 10.1109/CSF.2018.00022.

[20] Maha Kadadha, Hadi Otrok, Rabeb Mizouni, Shakti Singh, and Anis Ouali. On-chain behavior prediction machine learning model for blockchain-based crowdsourcing. *Future Generation Computer Systems*, 136:170–181, 2022. doi: 10.1016/j.future.2022.05.025.

[21] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, pages 3146–3154. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.

[22] Chris Lamb and Stefano Zacchiroli. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2):62–70, 2022. doi: 10.1109/MS.2021.3073045.

[23] Zhikai Li, Steve Vott, and Bhaskar Krishnamachari. ML2SC: Deploying machine learning models as smart contracts on the blockchain. In *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 645–649. IEEE, 2024. doi: 10.1109/ICBC59979.2024.10634431.

[24] Supun Nakandala, Karla Saur, Gyeong-In Yu, Konstantinos Karanasos, Carlo Curino, Markus Weimer, and Matteo Interlandi. A tensor compiler for unified machine learning prediction serving. In *14th USENIX Symposium on Operating Systems Design and Implementation*, pages 899–917. USENIX Association, 2020. ISBN 978-1-939133-19-9. URL https://www.usenix.org/conference/osdi20/presentation/nakandala.

[25] ONNX Working Group. Open neural network exchange intermediate representation specification. ONNX documentation, 2026. URL https://onnx.ai/onnx/repo-docs/IR.html. Accessed 24 July 2026.

[26] ONNX Working Group. `ai.onnx.ml` `TreeEnsemble` operator, version 5. ONNX operator documentation, 2026. URL https://onnx.ai/onnx/operators/onnx_aionnxml_TreeEnsemble.html. Accessed 29 July 2026.

[27] Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847.

[28] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alche Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program. *Journal of Machine Learning Research*, 22(164):1–20, 2021. URL https://jmlr.org/papers/v22/20-303.html.

[29] Marius Schlegel and Kai-Uwe Sattler. Capturing end-to-end provenance for machine learning pipelines. *Information Systems*, 132:102495, 2025. doi: 10.1016/j.is.2024.102495.

[30] scikit-learn developers. `sklearn.datasets.load_boston` documentation. scikit-learn 1.1 documentation, 2022. URL https://scikit-learn.org/1.1/modules/generated/sklearn.datasets.load_boston.html. Deprecated in 1.0 and scheduled for removal in 1.2; documentation describes ethical concerns. Accessed 25 July 2026.

[31] Nitin Singh, Pankaj Dayama, and Vinayaka Pandit. Zero knowledge proofs towards verifiable decentralized AI pipelines. In *Financial Cryptography and Data Security*, volume 13411 of *Lecture Notes in Computer Science*, pages 248–275. Springer, 2022. doi: 10.1007/978-3-031-18283-9_12. Financial Cryptography and Data Security 2022; proceedings published by Springer in 2023.

[32] Solidity Team. Solidity 0.8.20 documentation. Ethereum Foundation documentation, 2023. URL https://docs.soliditylang.org/en/v0.8.20/. Version-specific documentation; accessed 27 August 2026.

[33] Tobin South, Alexander Camuto, Shrey Jain, Shayla Nguyen, Robert Mahari, Christian Paquin, Jason Morton, and Alex Pentland. Verifiable evaluations of machine learning models using zkSNARKs. arXiv preprint, 2024. URL https://arxiv.org/abs/2402.02675.

[34] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1393–1410. USENIX Association, 2019. ISBN 978-1-939133-06-9. URL https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.

[35] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. Yellow paper, shanghai version, revision efc5f9a, Ethereum Project, February 2025. URL https://ethereum.github.io/yellowpaper/paper.pdf. Revision dated 4 February 2025. This cited revision specifies the Shanghai execution-layer version; later protocol forks are outside its scope.

[36] Jiaheng Zhang, Zhiyong Fang, Yupeng Zhang, and Dawn Song. Zero knowledge proofs for decision tree predictions and accuracy. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 2039–2053. ACM, 2020. doi: 10.1145/3372297.3417278.